

Opendesk: Grounding Desktop Computer-Use Agents via Native Accessibility APIs and Set-of-Marks Visual Prompting

Abhijith Neil Abraham¹ and Fariz Rahman¹

¹Vitalops, {abhijith, fariz}@vitalops.ai

Abstract

Most computer-use systems that deploy vision-language models (VLMs) for desktop automation ask the model to predict pixel coordinates directly from screenshots. This approach is brittle: coordinate predictions break under Retina/HiDPI scaling, window repositioning, zoom changes, and minor layout shifts. We present Opendesk, a desktop automation framework that addresses this by querying the platform’s native accessibility API before the screenshot reaches the model. Interactive elements are retrieved with their labels, roles, and bounding boxes, then rendered as numbered chips directly on the screenshot using the Set-of-Marks (SoM) visual prompting technique. The model selects elements by mark number rather than predicting coordinates, decoupling action selection from pixel-level localization. A secondary contribution is an adaptive replay mechanism: rather than replaying recorded coordinates, Opendesk stores trajectories as sequences of events and screenshots, then re-executes them by feeding the trajectory as context to the model, which re-grounds actions against the current screen state. Opendesk is released as an open-source Python library with a Model Context Protocol (MCP) server, supporting Anthropic, OpenAI-compatible, and LangChain agent harnesses.

1 Introduction

Automating desktop tasks using language models has become an active area of research following the introduction of multimodal models capable of interpreting screenshots [1, 5, 7]. The dominant approach is to present a screenshot to the model and ask it to output the (x, y) coordinates of the element to interact with. While this works in controlled settings, it introduces several failure modes in practice:

- Retina and HiDPI scaling: Physical pixel coordinates differ from logical coordinates by a scale factor (typically 2x on macOS Retina displays). Models trained on screenshots from mixed display environments may produce coordinates in the wrong space.
- Window repositioning: A coordinate that pointed to a button in the recorded state may point to empty space after the window is moved or resized.
- UI density and overlap: On dense UIs, nearby elements share similar visual neighborhoods, causing the model to select the wrong target.
- Layout changes: Application updates, dynamic content, or responsive layouts can shift elements by enough pixels to cause misclicks.

These failure modes share a common cause: the model is responsible for both identifying the target element and localizing it in pixel space. Separating these two responsibilities improves reliability considerably.

Opendesk addresses this by inserting a localization step before the screenshot reaches the model. The platform’s native accessibility API is queried to enumerate all interactive elements in the current window, returning their labels, roles, and bounding boxes in logical screen coordinates. These elements are rendered as numbered chips on the screenshot using the SoM technique [7]. The model now only needs to identify the correct mark number. Coordinate translation, Retina scaling, and window-relative offsets are handled deterministically by the framework.

This paper makes the following contributions:

1. A grounding architecture that combines native accessibility APIs with SoM visual prompting for reliable element targeting.
2. A UITool abstraction that targets elements by accessible label, promoting mouse coordinate control to a fallback.
3. An adaptive trajectory replay mechanism that re-grounds recorded workflows against the current screen state at replay time.
4. A three-layer open-source framework with a single Pydantic schema serving MCP, Anthropic SDK, OpenAI-compatible, and LangChain integrations simultaneously.

2 Related Work

Set-of-Marks (SoM) prompting. Yang et al. [7] introduced SoM as a technique for improving grounded visual reasoning in VLMs by overlaying alphanumeric labels on image regions. Subsequent work including OmniParser [4] and AgentS [1] applied SoM to UI screenshots. Opendesk extends this line by sourcing mark positions from the accessibility tree rather than a general-purpose vision detector, which gives exact bounding boxes for interactive elements rather than heuristic proposals.

Web and desktop automation. WebArena [9] and Mind2Web [2] established benchmarks for web agent evaluation. OSWorld [6] extended this to multi-application desktop tasks. These systems typically rely on HTML DOM structure (for web) or screenshot-only coordinate prediction (for desktop). Opendesk uses the OS-native equivalent of the DOM, the accessibility tree, for desktop targeting.

Accessibility-tree grounding. Concurrent work including WebVoyager [3] and SeeAct [8] has explored using the browser accessibility tree for web navigation. Opendesk applies the same principle to the full desktop across macOS, Linux, and Windows, and integrates it with SoM rendering.

RPA systems. Traditional robotic process automation tools such as UiPath and Automation Anywhere record workflows as sequences of selector-matched actions. Selectors are constructed from XPath or CSS patterns in the accessibility tree and break when the tree structure changes. Opendesk’s replay mechanism avoids selector brittleness by using the VLM to re-interpret the current screen state at replay time.

3 System Architecture

Opendesk is organized in three independent layers as shown in Figure 1.

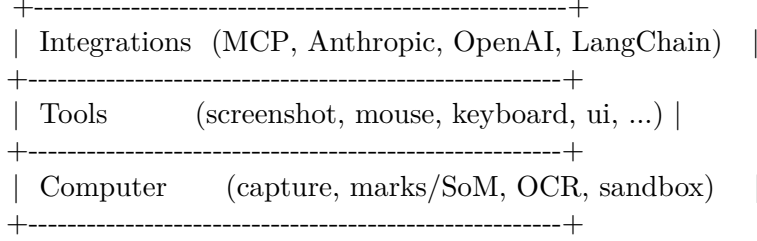


Figure 1: Three-layer architecture of Opendesk. Each layer is independently importable.

3.1 Layer 1: Computer

The computer layer provides low-level screen access with no dependency on the tools or integrations above it.

3.1.1 Screen Capture

Screen capture uses `mss` for cross-platform pixel access. `mss` returns pixels in BGRA order; PIL’s “BGRX” raw decoder reorders them to RGB. Screens wider than 1920 pixels are downsampled before the image is sent to the model to stay within LLM image size limits. A `diff_screenshots()` utility computes per-pixel differences using `ImageChops.difference()` and reports a change fraction and bounding box, useful for detecting whether an action had a visible effect.

3.1.2 Set-of-Marks Rendering

The `marks.py` module implements the SoM pipeline in two stages.

Stage 1: Element enumeration. `get_interactive_elements()` queries the native accessibility API:

- macOS: `osascript / AppleScript` queries System Events for AX roles including `AXButton`, `AXTextField`, `AXCheckBox`, and `AXMenuItem`.
- Linux: `pyatspi` walks the AT-SPI2 tree filtering by interactive roles.
- Windows: `pywinauto` UI Automation enumerates descendants() filtered by friendly class name.

Each element is returned as a record `{mark, role, label, x, y, w, h}` in logical screen coordinates.

Stage 2: Chip rendering. `draw_som_marks()` overlays numbered, color-coded chips on a PIL Image. Scale factors $s_x = W_{\text{screenshot}}/W_{\text{logical}}$ and $s_y = H_{\text{screenshot}}/H_{\text{logical}}$ translate logical coordinates to screenshot pixels, correctly handling Retina displays where $s_x = s_y = 2$.

3.1.3 OCR

Text extraction from screen regions uses three backends tried in priority order: (1) `pytesseract` for best cross-platform quality, (2) macOS Vision via a Swift subprocess requiring no additional dependencies on macOS 11+, and (3) Windows WinRT OCR via PowerShell requiring no additional dependencies on Windows 10+.

3.1.4 Sandbox and Audit

The `ComputerSandbox` class provides per-session policy enforcement and audit logging. Each action is recorded with a UUID, timestamp, parameters, and result. Optional constraints include an `allowed_apps` list and a `screen_region` bounding box; coordinates outside the bounding box are rejected before tool execution.

3.2 Layer 2: Tools

3.2.1 Core Abstractions

The `base.py` module defines four abstractions:

- `ToolResult`: title, output string, error flag, binary attachments, metadata dict.
- `Attachment`: filename, bytes content, MIME type.
- `ToolContext`: session identifier and an optional async permission handler called before every action.
- `Tool`: abstract base class with name, description, a Pydantic Params inner class, and an `execute()` coroutine.

`Tool.get__schema()` returns the JSON Schema for Params via Pydantic's `model__json__schema()`. This is the single source of truth consumed by all integration adapters, ensuring that tool definitions are never duplicated or allowed to diverge.

All blocking I/O runs in `asyncio.get_event_loop().run_in_executor(None, ...)` so tools are safe to call from async code without blocking the event loop.

3.2.2 UITool as Primary Interaction Path

A key design decision in `Opendesk` is that `UITool`, not `MouseTool`, is the primary interaction path. `UITool` finds elements by their accessible label using the platform's accessibility API and activates them directly, without translating to pixel coordinates at all. This means:

- No Retina scaling translation is required.
- The interaction succeeds regardless of window position or screen resolution.
- Failure messages are descriptive: "button 'Save' not found in `TextEdit`" rather than a silent misclick.

`MouseTool` is used only for elements that have no accessible label, such as canvas drawing areas, video players, and games.

3.3 Adaptive Trajectory Replay

3.3.1 Recording

`LearnRecorder` captures all user input globally via `pynput`, recording mouse moves, clicks, scroll events, and keystrokes, interleaved with screenshots at each discrete action. Each event is stored as a `TrajectoryEvent` dataclass with a timestamp, event type, parameters, and the associated screenshot. A `Trajectory` is a named, ordered sequence of `TrajectoryEvent` records persisted to `.opendesk/learned/` as JSON.

3.3.2 Replay

At replay time, Opendesk does not re-execute the recorded coordinate sequence. Instead, it:

1. Loads the stored Trajectory.
2. Builds a natural-language summary of the trajectory using `summary_builders` in `trajectory.py`.
3. Takes a fresh screenshot of the current screen state.
4. Submits the summary and current screenshot to the VLM, which identifies the corresponding action in the current UI.
5. Executes via `UITool` (label match) or `MouseTool` (coordinate fallback) as appropriate.

This design makes replay adaptive to UI changes. If an application was updated and a button moved, the replay succeeds as long as the button’s accessible label is unchanged. The model re-grounds each step independently rather than trusting that recorded coordinates remain valid.

3.4 Layer 3: Integrations

3.4.1 Model Context Protocol (MCP)

The MCP adapter wraps a `ToolRegistry` as an MCP Server. On `list_tools`, it returns tool definitions built from each tool’s JSON schema. On `call_tool`, it parses arguments, calls `tool.execute()`, and converts binary `Attachment` objects to `ImageContent` blocks for transmission over the MCP protocol.

3.4.2 Anthropic SDK

`ClaudeCodeAdapter` converts tool schemas to Anthropic’s `input_schema` format and drives the standard `stop_reason == "tool_use"` agentic loop. All `tool_use` blocks in a single model response are dispatched in parallel via `asyncio.gather`.

3.4.3 OpenAI-Compatible Endpoints

`OpenAIAdapter` wraps schemas in OpenAI’s `{"type": "function", "function": {...}}` envelope and handles `tool_calls` from `chat.completions.create` responses. Because it targets the OpenAI API format rather than the OpenAI service specifically, it is compatible with any conforming endpoint including Ollama, Groq, Together AI, vLLM, LiteLLM, and `llama.cpp` servers. This enables fully on-device computer-use agents with locally hosted models.

3.4.4 LangChain

`as_langchain_tools()` creates `BaseTool` subclasses dynamically from the registry, implementing both `__run` (sync) and `__arun` (async) entry points. The resulting tools are compatible with LangGraph’s `create_react_agent`.

3.5 Data Flow

The full request path from agent to action and back is:

1. The LLM outputs a tool name and argument dict.
2. `ToolRegistry.get(name)` retrieves the tool instance.
3. `Tool.parse_params(arguments)` validates via Pydantic.
4. `ToolContext.check_permission()` calls the injected policy handler.
5. `Tool.execute(ctx, params)` runs the action, calling into the computer layer and recording to the sandbox.
6. A `ToolResult` is returned and converted by the integration adapter into the format expected by the LLM client.

4 Discussion

4.1 Comparison with Coordinate-Prediction Approaches

Table 1 summarizes the behavioral differences between coordinate-prediction computer-use systems and Opendesk’s accessibility-grounded approach.

Property	Coordinate prediction	Opendesk (a11y + SoM)
Element targeting	Pixel coordinate output	Mark number selection
Retina handling	Model-dependent	Deterministic scale factor
Window repositioning	Breaks	Unaffected
UI layout change	Breaks	Adapts via label match
Unlabeled elements	Supported	Mouse fallback
Replay strategy	Coordinate replay	Trajectory re-grounding

Table 1: Comparison of coordinate-prediction and accessibility-grounded computer-use approaches.

4.2 Limitations

Several limitations apply to the current system.

Accessibility API coverage. Not all applications expose rich accessibility metadata. Games, Electron apps with custom renderers, and some cross-platform toolkits provide sparse or empty accessibility trees. In these cases Opendesk falls back to coordinate prediction, losing the reliability benefits of the grounded approach.

Dynamic content. Web applications that render content dynamically may produce accessibility trees that are stale or incomplete at query time. A snapshot taken before JavaScript completes rendering will be missing elements present at interaction time.

Replay semantic drift. If a recorded workflow involves elements whose accessible labels change between recording and replay, re-grounding will fail to match them. The model may recover by reasoning about visual similarity, but this is not guaranteed.

Formal evaluation. This paper presents a system description. Quantitative evaluation against OSWorld [6] or a similar benchmark is left for future work.

5 Conclusion

We presented Opendesk, a desktop automation framework that grounds VLM-driven computer-use agents via native accessibility APIs and Set-of-Marks visual prompting. By separating element identification from pixel-level localization, the system avoids the most common failure modes of coordinate-prediction approaches. The adaptive replay mechanism extends this principle to workflow recording and playback, re-grounding each step against the current screen state rather than trusting that recorded coordinates remain valid. Opendesk is released as open-source software at <https://github.com/vitalops/opendesk>.

References

- [1] Saaket Agashe, Jiuzhou Fan, Lukas Apodaca, Wentao Liao, Manling Li, and Xin Eric Wang. Agent s: An open agentic framework that uses computers like a human. arXiv preprint arXiv:2410.08164, 2024.
- [2] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. arXiv preprint arXiv:2306.06070, 2023.
- [3] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hong Zhang, Zhenzhong Lan, and Dong Yu. Webvoyager: Building an end-to-end web agent with large multimodal models. arXiv preprint arXiv:2401.13919, 2024.
- [4] Yadong Lu, Jianwei Li, Yelong Zhang, Yunsheng Song, Junda Li, Jianfeng Gao, and Weizhu Chen. Omniparser for pure vision based gui agent. arXiv preprint arXiv:2408.00203, 2024.
- [5] OpenAI. Gpt-4 technical report. Technical report, OpenAI, 2023.
- [6] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shi, Fangyu Liu, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. arXiv preprint arXiv:2404.07972, 2024.
- [7] Jianwei Yang, Hao Zhang, Feng Li, Xueyan Zou, Chunyuan Li, and Jianfeng Gao. Set-of-mark prompting unleashes extraordinary visual grounding in gpt-4v. arXiv preprint arXiv:2310.11441, 2023.
- [8] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. Gpt-4v(ision) is a generalist web agent, if grounded. arXiv preprint arXiv:2401.01614, 2024.
- [9] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. Webarena: A realistic web environment for building autonomous agents. arXiv preprint arXiv:2307.13854, 2023.